Project 2: Constrained Optimization

Daniel T. Simpson datsimp@stanford.edu
AA222/CS361, Stanford University

1. Algorithm Description

When I first read the open problems, Simple1, Simple2, and Simple3, I noticed that each only had inequality constraints, no equality constraints. So, I chose a robust method that did not require ρ to approach infinity (which is a risk for unadjusted, simple penalty methods).

1.1 Augmented Lagrangian Method (ALM)

I chose to implemented an augmented Lagrangian method (ALM), also known as a Method of Multipliers. The augmented function is shown in Equation 1 where $f(\mathbf{x})$ is the function to optimize and ρ is the weighing coefficient of the penalty function, $p_{\text{Lagrange}}(\mathbf{x})$.

$$\underset{\mathbf{x}}{\text{minimize}} f(\mathbf{x}) + \rho \cdot p_{\text{Lagrange}}(\mathbf{x}) \tag{1}$$

ALM defines the penalty function by combining a quadratic term on the constraint function, and a linear term on both the constraint function and the respective Lagrange multiplier (Kochenderfer and Wheeler (2019)). The constraint function, $c(\mathbf{x})$, is either an equality constraint, $h(\mathbf{x})$, or inequality constraint, $g(\mathbf{x})$. The Lagrange multiplier is updated based on the weight on the penalty and the constraint function. For equality constraints, this is shown in Equations 2 and 3.

$$p_{\text{Lagrange}}(\mathbf{x}) = \frac{1}{2}\rho \sum_i (h_i(\mathbf{x}))^2 + \sum_i \lambda_i h_i(\mathbf{x}) \tag{2}$$

$$\lambda^{(k+1)} = \lambda^{(k)} + \rho \mathbf{h}(\mathbf{x}^{(k+1)}) \tag{3}$$

However, inequality constraints require different augmented equations. Unlike equality constraints which are inactive only when the constraint function is satisfied, inequality constraints can be inactive for subsets of the search domain, called feasible sets. Thus, Equation 2 can be replaced by the penalty function for inequalities by Bertsekas (1996), and Equation 3 and be adjusted to account for feasible sets.

$$p_{\text{Lagrange}}(\mathbf{x}) = \frac{1}{2\rho} \sum_i (\max[0, \mu_i + \rho \mathbf{c}_i(\mathbf{x})]^2 - \mu_i^2) \tag{4}$$

$$\mu^{(k+1)} = \mu^{(k)} + \rho \max[\mathbf{c}(\mathbf{x}^{(k+1)}), 0] \tag{5}$$

1.2 Quadratic Penalty Method (QPM)

Once I was able to get ALM working with an ADAM optimizer (see next section), I decided to also try a simple quadratic penalty function. The quadratic penalty function works by

finding the sum off all violated constraints squared according to Equation 6.

$$p_{\text{Quadratic}}(\mathbf{x}) = \sum_i \max[\mathbf{c}(\mathbf{x}), 0]^2 \tag{6}$$

This replaces p_{Lagrange} in Equation 1 to define the augmented function as:

$$\underset{\mathbf{x}}{\text{minimize}} f(\mathbf{x}) + \rho \cdot p_{\text{Quadratic}}(\mathbf{x}) \tag{7}$$

1.3 ADAM

From Project1, I knew the Adam (Adaptive Moment Estimation) optimizer would be a robust choice for optimizing the augmented function, $f(\mathbf{x}) + \rho \cdot p_{\text{Lagrange}}(\mathbf{x})$. It tracks the exponentially decaying average of past gradients (first moment, $\mathbf{v}$) and past squared gradients (second raw moment, $\mathbf{s}$). These are updated using decay rates γ_v and γ_s, as described in Equations 6 and 7.

$$\mathbf{v}^{(k)} = \gamma_v \mathbf{v}^{(k-1)} + (1 - \gamma_v)\hat{\mathbf{g}}^{(k)} \tag{8}$$

$$\mathbf{s}^{(k)} = \gamma_s \mathbf{s}^{(k-1)} + (1 - \gamma_s)(\hat{\mathbf{g}}^{(k)})^2 \tag{9}$$

This method requires the gradient of the augmented function, which I calculated by ensuring the augmented function is continuous and using a forward, first-order, finite difference method according to Equation 8.

$$\hat{\mathbf{g}}_i(\mathbf{x}) = \frac{\mathcal{L}(\mathbf{x} + h\mathbf{e}_i) - \mathcal{L}(\mathbf{x})}{h} \tag{10}$$

where $h = 10^{-5}$ is the step size and $\mathbf{e}_i$ is the i-th unit vector. Finally, the variables are adjusted to counteract bias, as shown in Equations 9 and 10.

$$\hat{\mathbf{v}}^{(k)} = \frac{\mathbf{v}^{(k)}}{1 - \gamma_v^k} \tag{11}$$

$$\hat{\mathbf{s}}^{(k)} = \frac{\mathbf{s}^{(k)}}{1 - \gamma_s^k} \tag{12}$$

The design point is updated using the bias-corrected moments, a small constant ϵ to prevent division by zero, and the decaying step size $\alpha^{(k)}$ (Equation 12).

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha^{(k)} \frac{\hat{\mathbf{v}}^{(k)}}{\sqrt{\hat{\mathbf{s}}^{(k)}} + \epsilon} \tag{13}$$

Finally, I would update the decaying step size according to, $\alpha^{(k+1)} = \gamma_\alpha \alpha^{(k)}$. It is also worth noting that between Adam optimization method calls, I would preserve the α, $\mathbf{v}$, $\mathbf{s}$ parameters and iteration index k from the prior Adam method call.

1.4 Results

Both methods successfully achieve ¿95% feasibility on all tested problems. I chose to apply ALM to problems Simple1 Secret1, and Secret2 since these problems either have steep gradients that ALM was better at optimizing, or high dimensionality which ALM is able to better find the feasible space of. I chose to apply QPM to Simple2 and Simple3 because when running tests I found it had better results than with ALM. This is likely because ALM over penalized the constraints and pushed the design point away from the optimum near the constraint boundary.

2. Choosing Hyperparameters

After much trial and error, I arrived at a set of hyperparameters that would lead to successful optimization algorithms. These hyperparameters are summarized in Table 1 and their definitions are summarized in Table 2.

Table 1: Chosen Hyperparameters for Constrained Problems

Parameter	**Simple1**	**Simple2**	**Simple3**	**Secret1**	**Secret2**
Method	ALM	QPM	QPM	ALM	ALM
ρ	1.0	1.0	1.0	1.0	1.0
γ_ρ	1.50	1.50	1.50	1.50	1.50
$N_{\text{Inner Loops}}$	12	12	12	3	2
α_0	0.50	0.10	0.50	0.50	0.50
γ_α	0.98	0.99999	0.99	0.99	0.95
γ_v	0.9	0.9	0.9	0.9	0.9
γ_s	0.999	0.999	0.999	0.999	0.999
ϵ	1e-8	1e-8	1e-8	1e-8	1e-8

I decided to set the parameters for Adam, γ_α, γ_v, γ_s, and ϵ to be equal to what worked in Project 1. I then adjusted $N_{\text{Inner Loops}}$, α_0, and γ_α depending on whether the problem had steep gradients or not. The tuning process went as such:

1. **Simple1**: I started off with $\alpha_0 = 0.65$, a large step size, and a decay rate of $\gamma_\alpha = 0.95$. I also chose to give Adam $N_{\text{Inner Loop}} = 12$ steps so it could have sufficient steps to arrive at an optimum along the constraint boundary. However, when looking at plots the paths would take when trying to optimize, I noticed their step sizes were too large. So, I decreased the initial step size and slowed the rate of decay. This led to successful results.

2. **Simple2**: I carried the values from Simple1 to Simple2, which had much steeper gradients. This led to unsuccessful results, so I decided to further drop the step size and significantly decrease the rate of step decay. Since Adam parameters would carry over from one iteration to another within the ALM or QPM, the algorithm could

Table 2: Hyperparameters Definitions

Symbol	Description
k	Iteration index
ρ	Penalty weight
γ_p	Growth rate for penalty weight
$N_{\text{Inner Loops}}$	The number of loops Adam can internally run
α_0	Initial step size
γ_α	Decay rate for α
γ_v	Bias factor for $\mathbf{v}$
γ_s	Bias factor for $\mathbf{s}$
ϵ	Buffer & Resolution termination threshold

take many small steps towards the optimum, balancing between moving along the constraint boundary or moving towards the overall function optimum. This led to successful results

3. **Simple3**: I simply applied the same parameters from Simple1 to Simple3 and found great success. No adjustment was needed.

4. **Secret1**: With the larger dimension spaces that come with the secret problems, I decided to drop the number of inner loops from 12 to 3 to bias the algorithm to optimizing along constraints rather than along the optimal path. Since the actual function and the actual constraints were unknown, this was a safe way to preserve compute towards reaching the feasible region.

5. **Secret2**: I decided to drop the number of inner loops to 2 to preserve more computation towards reaching the feasible set. I also found that the algorithm would take too long to run with a step decay rate of 0.99 like with prior problems so I dropped it to 0.95.

3. Explaining the Algorithm

Both methods operate with two nested loops: one loop that updates the penalty and an inner loop that minimizes the augmented objective using ADAM.

3.1 Outer Loop (ALM)

For each outer iteration, the algorithm minimizes the objective $f(\mathbf{x}) + \rho \cdot p_{\text{Lagrange}}(\mathbf{x})$ to get $\mathbf{x}^{(k+1)}$. Then, it updates the Lagrange multipliers $\mu^{(k+1)}$ and increases the penalty weight $\rho^{(k+1)}$. The method converges to a point that satisfies the KKT conditions, as seen with the align the function gradients and active constraint gradient. When the constraint is inactive, the algorithm optimizes within the feasible set, similar to an unconstrained function, without considering constraint boundaries Intuitively, this means the algorithm tries to optimize the

objective value function while balancing the constraints. If the constraints are violated, then the value of μ increases and stays higher for all future outer loop iterations. The $\mu_i c_i(\mathbf{x})$ term pulls the function away from violating the constraints. In a way, it's as if the algorithm has memory over its violated constraints. This allows ALM to find the optimum without requiring ρ to approach infinity. The quadratic penalty term $\frac{\rho}{2} c_i(\mathbf{x})^2$ turns the function into a bowl that ensures there is a defined minimum. Whereas a pure Lagrangian can be unbounded causing the optimizer to diverge, the quadratic term grows rapidly as ρ grows. Thus, ALM balances minimizing $f(\mathbf{x})$ and driving $c(\mathbf{x})$ to zero.

3.2 Outer Loop (QPM)

For each outer iteration, the algorithm minimizes the objective $f(\mathbf{x}) + \rho \cdot p_{\text{Quadratic}}(\mathbf{x})$ to get $\mathbf{x}^{(k+1)}$. Then, it increases the penalty weight $\rho^{(k+1)}$. Unlike ALM, which has a penalty function dependent on a Lagrangian multiplier, QPM simply squares the constraint value. This memory-less approach is simply to implement and achieves the same constraint satisfaction as ALM, but requires larger ρ values.

3.3 Adam

Adam treats the incoming functions as an unconstrained optimization problem and works by tracking the gradient and squared gradient across iterations similar to momentum, AdaGrad, and RMSProp methods. The gradient and squared gradients decay across iterations, allowing for large steps in low-gradient regions and small steps in high-gradient regions. This combination helps guarantee a smooth and quick descent.

4. Pros and Cons

4.1 Pros

1. ALM is parameter-efficient: Only γ_α and $N_{\text{number of loops}}$ and α_0 needed tuning.
2. ALM converges with lower values of ρ: The memory feature of ALM removes the need to many steps and for $\rho \to \infty$, unlike other penalty methods.
3. QPM is simple: There are fewer hyperparameters to implement
4. Both methods are robust with Adam: Adam has adaptive step sizing that allows for quick, smooth minimizations.

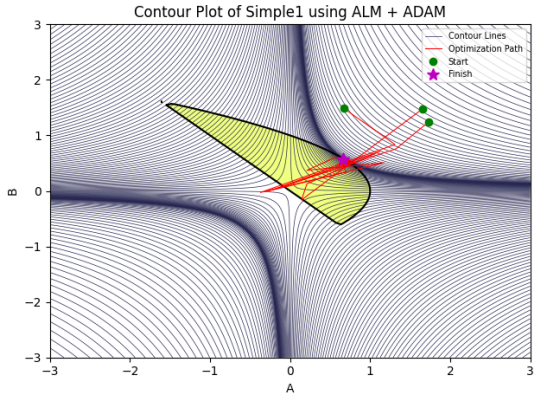
4.2 Cons

1. ALM has a high per-iteration cost: ALM computes the gradient with $d+1$ constraint evaluations per Adam step, where d is the problem dimension, making its processing time long.
2. QPM requires many outer loop iterations: It does not retain memory of constraint violations. Thus, it takes many outer loop steps for ρ to sufficiently amplify constraint conditions.

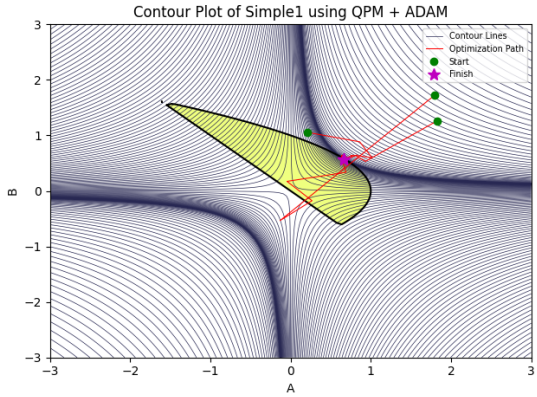
3. Both methods are not guaranteed to find the global minimum: the final solution depends heavily on $\mathbf{x}_0$ since the methods are gradient step methods.

5. Plots

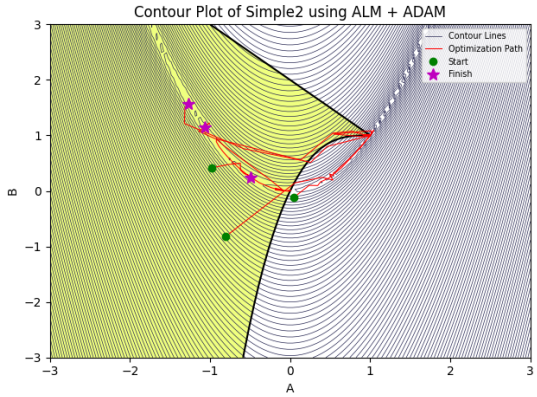
In Figure 1 we see that both methods easily converge to a single point for Simple1, but for Simple2, spread across a valley of minimum points. The objective value plots in Figure 2 show that the algorithms converge within 20 steps, but become unstable with more iterations. This suggests that as the algorithm continues to attempt to converge to more precise minimums, numerical instability pushes the design point away from the best minimum. We see this with the maximum constraint violation plots shifting away from the boundary as ρ increases.



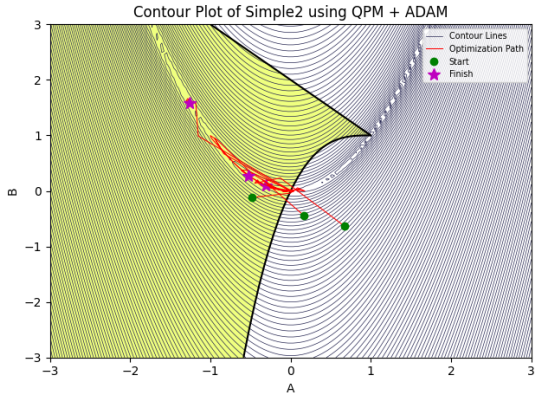
(a) Simple1: Augmented Lagrangian



(b) Simple1: Quadratic Penalty

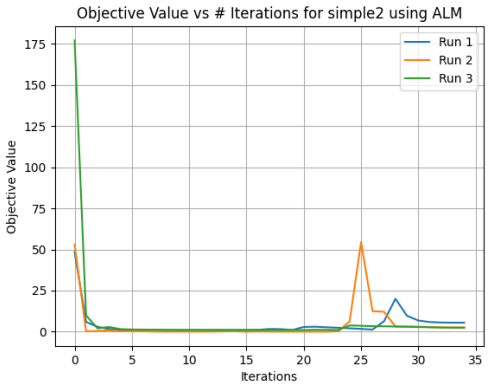


(c) Simple2: Augmented Lagrangian

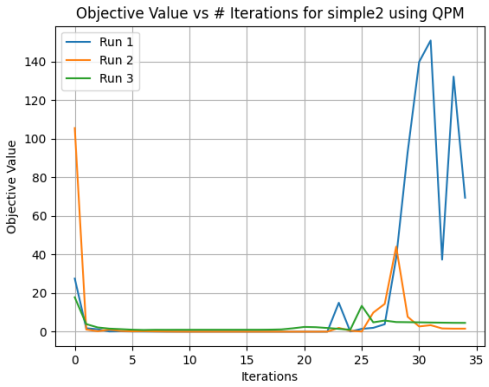


(d) Simple2: Quadratic Penalty

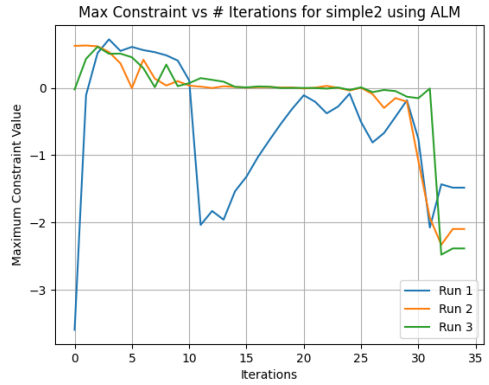
Figure 1: Contour plots comparing ALM and QPM optimization paths for Simple1 and Simple2. The yellow highlighted region is the feasible region of $c(\mathbf{x}) \leq 0$



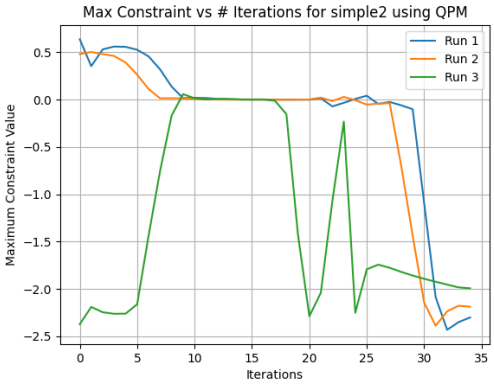
(a) Simple2: Objective Value vs Iteration using ALM



(b) Simple2: Objective Value vs Iteration using QPM



(c) Simple2: Maximum Constraint Value vs. Iteration using ALM



(d) Simple2: Maximum Constraint Value vs. Iteration using QPM

Figure 2: Plots comparing ALM and QPM objective values and constraint values for Simple2.

References

Dimitri P. Bertsekas. *Constrained Optimization and Lagrange Multiplier Methods*, chapter 3. Athena Scientific, Belmont, MA, 1996. ISBN 1-886529-04-3.

Mykel J. Kochenderfer and Tim A. Wheeler. *Algorithms for Optimization*. MIT Press, Cambridge, Massachusetts, 2019. ISBN 9780262039420.

project2_py\project2.py

```python
#
# File: project2.py
#

## top-level submission file

'''
Note: Do not import any other modules here.
        To import from another file xyz.py here, type
        import project2_py.xyz
        However, do not import any modules except numpy in those files.
        It's ok to import modules only in files that are
        not imported here (e.g. for your plotting code).
'''
import numpy as np

try:
    from .helpers import Simple1, Simple2, Simple3
    from .local_minimizers import adam_optimizer
except ImportError:
    from helpers import Simple1, Simple2, Simple3
    from local_minimizers import adam_optimizer


def augmented_lagrange_ADAM(f, g, c, x0, n, count, path, params = None):
    #Hyper terms
    x_length = len(x0)

    # ALM Terms
    if count() < n:
        c0 = np.array(c(x0)).flatten() # Costs 1
        mu = np.zeros(len(c0))
    else:
        mu = np.array([])
    rho = params.get('rho', 1.0)
    gamma_rho = params.get('gamma_rho', 1.01)
    num_inner_loops = params.get('num_inner_loops', 12)

    #ADAM Terms
    alpha = params.get('alpha', 0.32)
    gamma = params.get('gamma', 0.95)
    gamma_v = params.get('gamma_v', 0.9)
    gamma_s = params.get('gamma_s', 0.999)
    epsilon = params.get('epsilon', 1e-5)

    def penalty(x, mu, rho, c): # Each penalty call costs 1
        if len(mu) == 0:
            return 0.0
        constraints = np.array(c(x)).flatten()
        return float(np.sum((np.maximum(mu + rho*constraints, 0)**2 - mu**2) / (2*rho)))

    def penalty_g(x, mu, rho, c, h = 1e-5): # Each penalty_g call costs 1 + x_length
```

```
        x_length = len(x)
        penalty_gh = np.zeros(x_length)
        if len(mu) == 0:
            return penalty_gh
        p0 = penalty(x, mu, rho, c)
        for i in range(x_length):
            x_h = np.copy(x)
            x_h[i] += h
            penalty_gh[i] = penalty(x_h, mu, rho, c) - p0 # Forward Diff Gradient Approx.
        return penalty_gh / h

    vel = np.zeros(x_length)
    sqr = np.zeros(x_length)
    k = 0
    while count() < n:

        x_old = path[-1]

        augmented_f = lambda x: f(x) + penalty(x, mu, rho, c) # Each augmented_f call costs 2
        augmented_g = lambda x: g(x) + penalty_g(x, mu, rho, c) # Each augmented_g call costs 3 +
x_length

        cost_g = 3 + x_length if len(mu) > 0 else 2
        evals_for_mu = 1 if len(mu) > 0 else 0
        augmented_n = np.minimum(count() + cost_g * num_inner_loops, n - evals_for_mu)

        adam_path, alpha, k, vel, sqr = adam_optimizer(
            augmented_f, augmented_g, x_old, augmented_n, count,
            alpha, gamma, gamma_v, gamma_s, epsilon,
            k, vel, sqr, cost_g
        )

        x_new = adam_path[-1]
        if len(mu) > 0 and count() < n:
            mu = np.maximum(mu + rho * np.array(c(x_new)).flatten(), 0) # Update the Lagrange
multiplier (This costs 1)
        path.append(x_new)
        rho = rho * gamma_rho

        if np.linalg.norm(x_new - x_old) < epsilon:
            break
    return path



def quadratic_penalty_ADAM(f, g, c, x0, n, count, path, params = None):
    if params is None:
        params = {}
    x_length = len(x0)

    rho = params.get('rho', 1.0)
    gamma_rho = params.get('gamma_rho', 1.01)
    num_inner_loops = params.get('num_inner_loops', 12)

```

```
    #ADAM Terms
    alpha = params.get('alpha', 0.30)
    gamma = params.get('gamma', 0.999)
    gamma_v = params.get('gamma_v', 0.9)
    gamma_s = params.get('gamma_s', 0.999)
    epsilon = params.get('epsilon', 1e-5)

    vel = np.zeros(x_length)
    sqr = np.zeros(x_length)
    k = 0

    def penalty(x, rho, c): # Each penalty call costs 1
        constraints = np.array(c(x)).flatten()
        return (rho / 2.0) * np.sum(np.maximum(constraints, 0)**2)

    def penalty_g(x, rho, c, h = 1e-5): # Each penalty_g call costs 1 + x_length
        x_length = len(x)
        penalty_gh = np.zeros(x_length)
        p0 = penalty(x, rho, c)
        for i in range(x_length):
            x_h = np.copy(x)
            x_h[i] += h
            penalty_gh[i] = penalty(x_h, rho, c) - p0 # Forward Diff Gradient Approx.
        return penalty_gh / h

    augmented_f = lambda x: f(x) + penalty(x, rho, c) # Each augmented_f call costs 2
    augmented_g = lambda x: g(x) + penalty_g(x, rho, c) # Each augmented_g call costs 3 + x_length

    while count() < n:
        x_old = path[-1]
        cost_g = 3 + x_length
        augmented_n = np.minimum(count() + cost_g * num_inner_loops, n)

        adam_path, alpha, k, vel, sqr = adam_optimizer(
            augmented_f, augmented_g, x_old, augmented_n, count,
            alpha, gamma, gamma_v, gamma_s, epsilon,
            k, vel, sqr, cost_g
            )
        x_new = adam_path[-1]
        path.append(x_new)
        rho = rho * gamma_rho
        if np.linalg.norm(x_new - x_old) < epsilon:
            break
    return path



def optimize_with_history(f, g, c, x0, n, count, prob):
    """
    Args:
        f (function): Function to be optimized
        g (function): Gradient function for `f`
        c (function): Function evaluating constraints
        x0 (np.array): Initial position to start from
```

```
        n (int): Number of evaluations allowed. Remember `f` and `c` cost 1 and `g` costs 2
        count (function): takes no arguments are reutrns current count
        prob (str): Name of the problem. So you can use a different strategy
                 for each problem. `prob` can be `simple1`,`simple2`,`simple3`,
                 `secret1` or `secret2`
    Returns:
        x_best (np.array): best selection of variables found
    """

    path = [x0]
    if prob == 'simple1':  ### Augmented Lagrange Method (ALM) ###
        #return []
        simple1_params = {
            'rho': 1.0,
            'gamma_rho': 1.50,
            'num_inner_loops': 12,
            'alpha': 0.50,
            'gamma': 0.98,
            'gamma_v': 0.9,
            'gamma_s': 0.999,
            'epsilon': 1e-8
            }
        #path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, simple1_params)
        path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, simple1_params)

    elif prob == 'simple2':
        #return []
        simple2_params = {
            'rho': 1.0,
            'gamma_rho': 1.5,
            'num_inner_loops': 12,
            'alpha': 0.10,
            'gamma': 0.99999,
            'gamma_v': 0.9,
            'gamma_s': 0.999,
            'epsilon': 1e-8
            }
        #path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, simple2_params)
        path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, simple2_params)

    elif prob == 'simple3':
        #return []
        simple3_params = {
            'rho': 1.0,
            'gamma_rho': 1.50,
            'num_inner_loops': 12,
            'alpha': 0.50,
            'gamma': 0.99,
            'gamma_v': 0.9,
            'gamma_s': 0.999,
            'epsilon': 1e-8
            }
        path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, simple3_params)
        #path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, simple3_params)
```

```python

    elif prob == 'secret1':
        #return []
        secret1_params = {
            'rho': 1.0,
            'gamma_rho': 1.50,
            'num_inner_loops': 3,
            'alpha': 0.50,
            'gamma': 0.99,
            'gamma_v': 0.9,
            'gamma_s': 0.999,
            'epsilon': 1e-8
            }
        path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, secret1_params)
        #path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, secret1_params)

    elif prob == 'secret2':
        #return []
        secret2_params = {
            'rho': 1.0,
            'gamma_rho': 1.50,
            'num_inner_loops': 2,
            'alpha': 0.50,
            'gamma': 0.95,
            'gamma_v': 0.9,
            'gamma_s': 0.999,
            'epsilon': 1e-8
            }
        #path = augmented_lagrange_ADAM(f, g, c, x0, n, count, path, secret2_params)
        path = quadratic_penalty_ADAM(f, g, c, x0, n, count, path, secret2_params)

    return path



def optimize(f, g, c, x0, n, count, prob):
    """
    Wrapper for autograder that only returns the final point.
    """
    path = optimize_with_history(f, g, c, x0, n, count, prob)
    return path[-1]

if __name__ == '__main__':
    try:
        from .plotters import plot_problem, plot_convergence, plot_constraint
    except ImportError:
        from plotters import plot_problem, plot_convergence, plot_constraint
    current_problem = Simple2()
    if current_problem.xdim == 2:
        print(f"Rendering 2D optimization paths for {current_problem.prob}...")
        plot_problem(current_problem, plot_size=3)
        plot_convergence(current_problem)
        plot_constraint(current_problem)

```

project2_py\local_minimizers.py

```python
import numpy as np

def adam_optimizer(f, g, x0, n, count, alpha, gamma, gamma_v, gamma_s, epsilon, k=0, vel=None, sqr=None, cost_g=None):
    path = [x0]
    x_best = x0
    x_len = len(x0)
    if sqr is None: sqr = np.zeros(x_len)
    if vel is None: vel = np.zeros(x_len)
    if cost_g is None: cost_g = 3 + x_len

    while count() + cost_g <= n:
        k += 1
        gradient = g(x_best) # Costs
        vel = gamma_v*vel + (1-gamma_v)*gradient
        sqr = gamma_s*sqr + (1-gamma_s)*(gradient**2)
        vel_hat = vel / (1 - gamma_v**k)
        sqr_hat = sqr / (1 - gamma_s**k)
        step = alpha * vel_hat / (epsilon + np.sqrt(sqr_hat))
        x_best = x_best - step
        path.append(x_best)
        alpha *= gamma
        if np.linalg.norm(step) < epsilon:
            break
    return path, alpha, k, vel, sqr
```